

FlavorMap: Restaurant Discovery and Review Platform

AUTHORS

Mardin Nasouti (241401908)
Arsha Sharify Fakher (241401901)
Tugra Duygulu (241401022)
Burak Demirköz (241401022)

Supervisor Prof. Ahmet Bulut
University Acibadem University
Dept Computer Engineering (CSE)
Repo  [krr0ption/cse-220](https://github.com/krr0ption/cse-220)

ABSTRACT

FlavorMap is a restaurant discovery and review system built with Django 6 on the backend and Next.js 16 on the frontend. The project covers restaurant management, public discovery, authenticated reviews and reactions, one-level review replies, owner-facing analytics, and file-backed restaurant photos. The implementation follows the staged backlog documented in [docs/PRODUCT_REQUIREMENTS.md](#), [docs/DEVELOPMENT_PLAN.md](#).

The project utilizes  for monorepo management, with separate apps for the backend API and frontend UI, plus a shared library for authentication utilities. The monorepo architecture supports clear separation of concerns, code reuse, and streamlined development workflows across the full stack.

The final report details the system architecture, data model, feature implementation checklist, challenges and solutions, team communication, and individual contributions.

1. Introduction

FlavorMap was designed as a practical full-stack web application for restaurant discovery and feedback collection. The user-facing flow supports diners who want to search restaurants, inspect ratings, read reviews, and react to useful feedback. The owner-facing flow supports authenticated restaurant owners who manage their own listings, monitor average ratings, and answer reviews as official business responses.

The project scope in the documentation was intentionally incremental. The PRD split the work into infrastructure, core restaurant data, review interactions, discovery, and map-driven features, while the development plan sequenced implementation so the app could remain demoable after each milestone. That structure is reflected in the merged PR history and in the discovery-review issue set, which decomposed the review experience into nine smaller tasks.

2. System Architecture

The system uses a layered architecture:

- The Django API is built with Django REST Framework (, DRF), which provides serializers, viewsets/API views, authentication hooks, permissions, request validation, and REST response handling.
- The Next.js client renders the public discovery pages, restaurant detail pages, and owner dashboard.
- Shared UI primitives in  provide cards, tabs, charts, forms, and layout components.
- File storage is abstracted so restaurant photos can be uploaded through a consistent API while the backend decides where the asset lives.

DRF is the boundary between the frontend and backend: the client sends JSON requests, DRF validates and serializes the data, and the service/repository layer performs the domain operations. The backend exposes REST endpoints for restaurants, reviews, categories, files, and owner-specific analytics. The owner dashboard uses a dedicated endpoint for summary metrics, per-restaurant reviewer stats, and monthly rating progress, which keeps analytics logic server-side and avoids duplicating aggregation rules in the browser.

The frontend architecture is centered around a route-based composition model. Public discovery pages load restaurant search results and selected restaurant review threads. The owner dashboard reuses the same restaurant and review data model, but layers on editing, response workflows, and analytics visualizations.

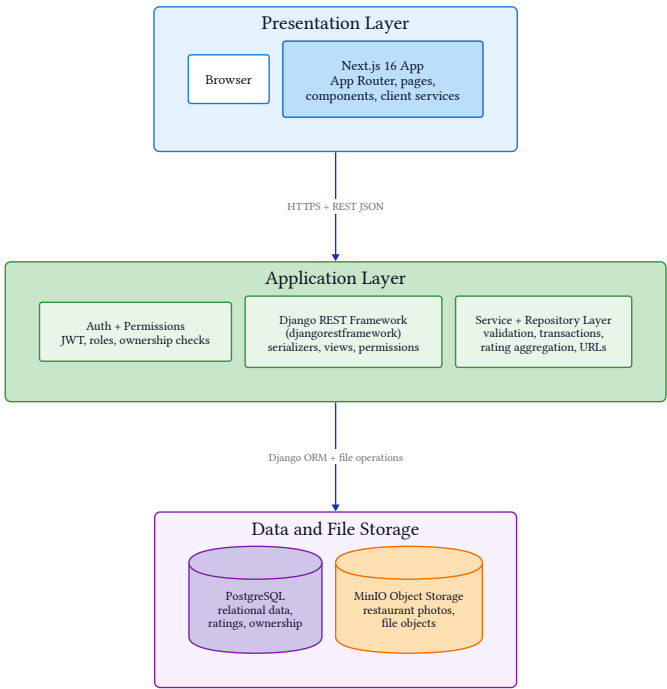
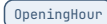


Figure 1: Overall system architecture showing the Django API and Next.js client.

3. Data Model

The implemented data model follows the product requirements closely:

-  stores the public listing, ownership, address, rating aggregate, and photo reference.
-  organizes restaurants by cuisine or concept.
-  describes weekly operating hours for the listing.

- `MenuItem` represents editable menu entries tied to a restaurant and category.
- `Review` stores diner ratings, written feedback, and one-level replies.
- `ReviewLike` and the dislike counterpart capture helpfulness reactions and support sorting by engagement.
- File storage entities and services manage uploaded images and generate safe asset URLs.

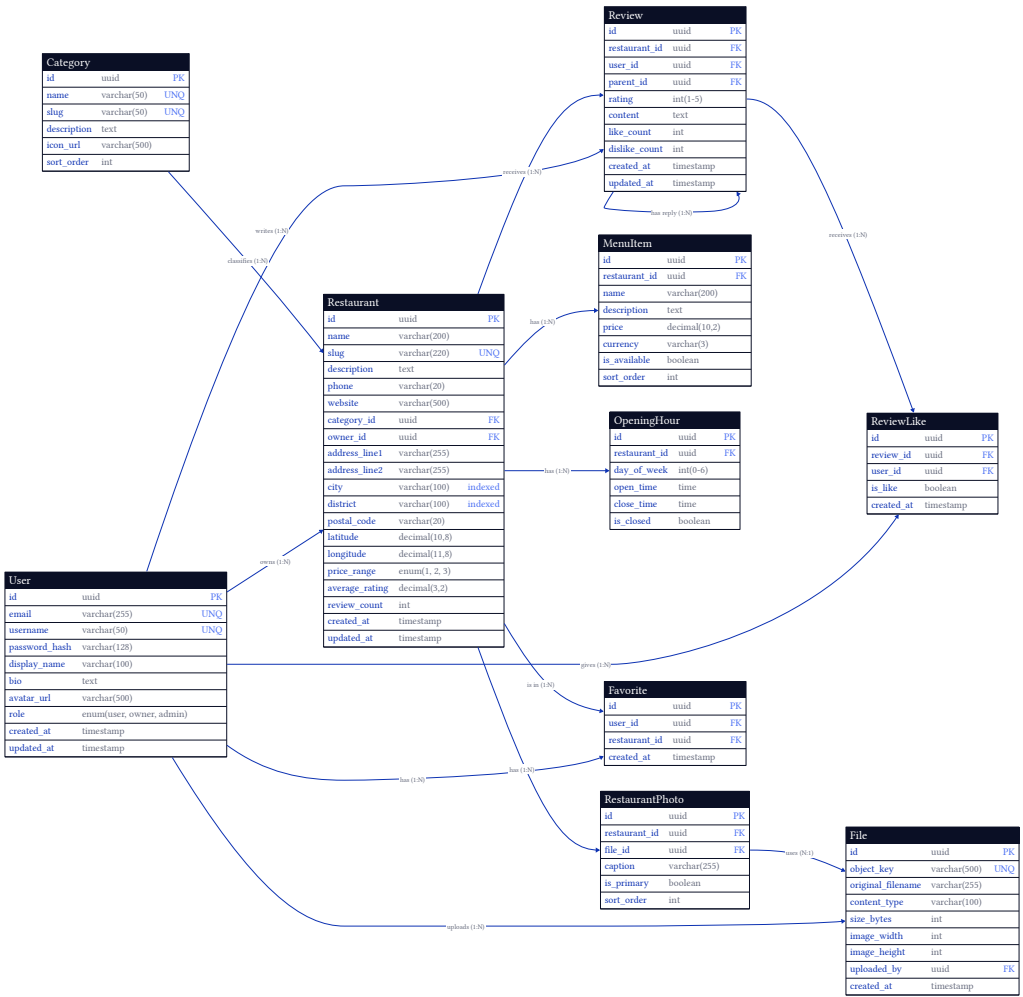


Figure 2: Database schema showing the main FlavorMap entities and relationships.

The aggregation logic is concentrated in the repository layer so that the API can expose stable totals without making the frontend re-derive rating statistics. That matters for owner analytics because the dashboard needs three separate views of the same data: restaurant-level average rating, reviewer-level behavior, and monthly progress over time.

4. App Demo

The following screenshots demonstrate the main implemented application flows: public discovery, map-based exploration, filtering, and owner menu management.

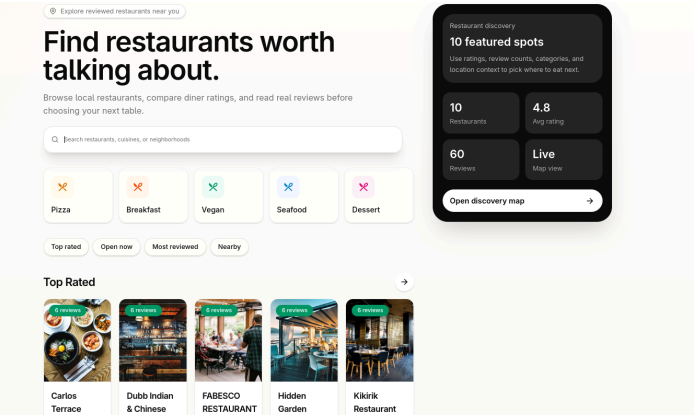


Figure 3: FlavorMap home page with public restaurant discovery entry points.

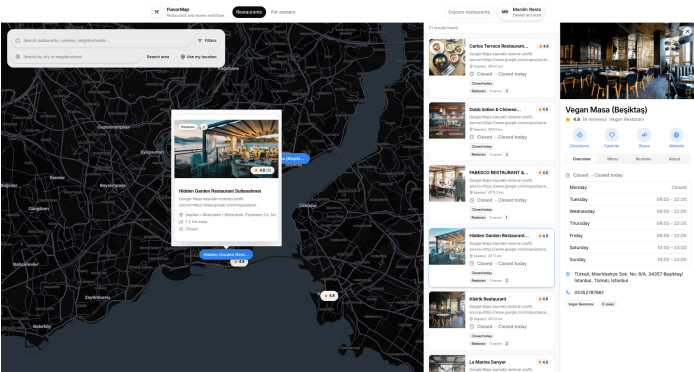


Figure 4: Restaurant exploration view combining listing results with map context.

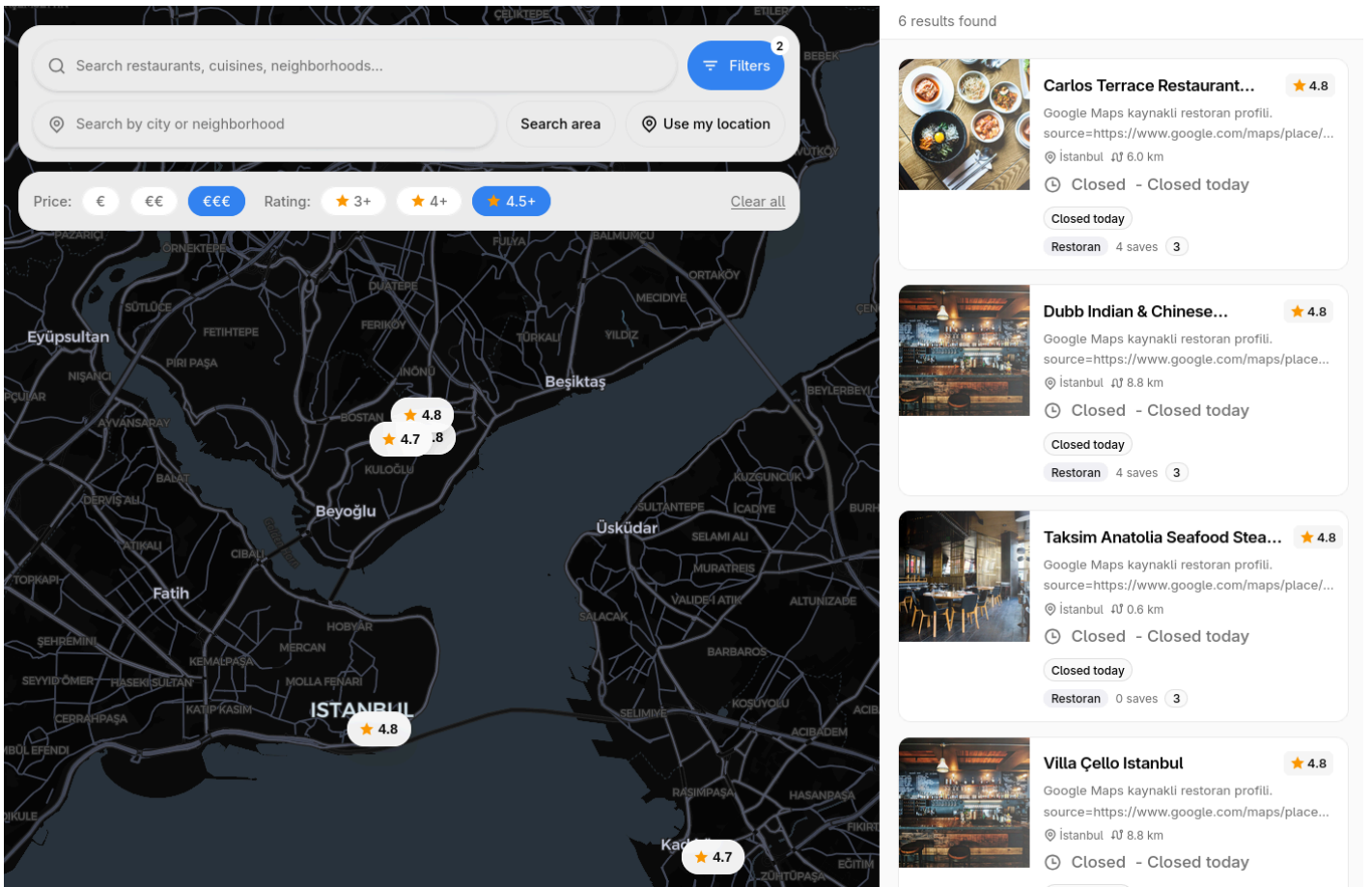


Figure 5: Filtering interface for narrowing restaurant results by user preferences.

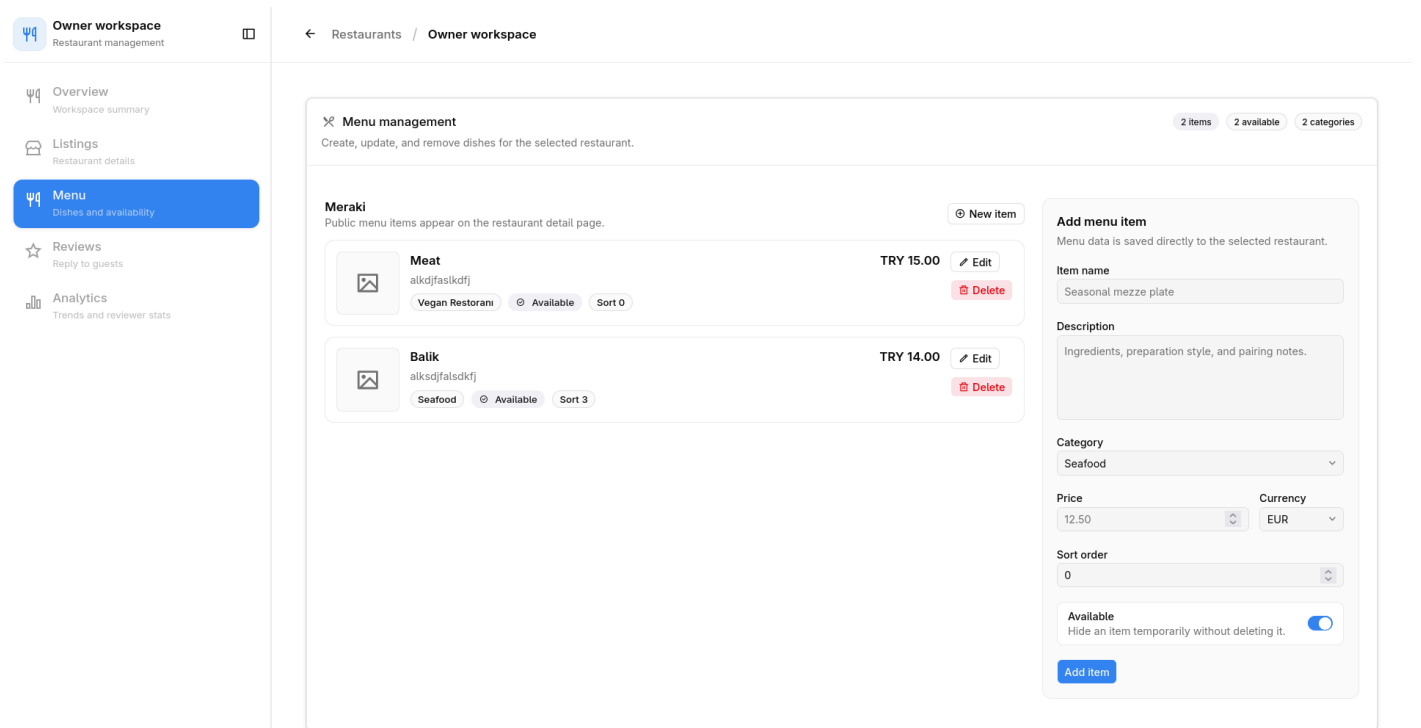


Figure 6: Owner dashboard menu management flow for maintaining restaurant menu items.

5. Feature Checklist

The issue tracker is the source of truth for the feature checklist, while merged pull requests are the source of truth for implemented contributions.

This checklist is partitioned by GitHub epic. Each epic has its own table with completion status, tracker item, feature name, and the implementation evidence used for the final report.

Epic: Restaurant CRUD and Category Management ([Closed Issue #2](#))

Done	Tracker	Feature	Description and PR evidence
✓	Closed Issue #38	Restaurant Listing and Management	Implemented through Merged PR #40 and Merged PR #48 , establishing restaurant listing, owner management, DRF API structure, serializers, service/repository boundaries, and admin-dashboard foundations.
✓	Closed Issue #3	Category System	Implemented through Merged PR #56 , migrating restaurants from a single category to a many-to-many category model and updating serializers, admin filters, seed data, and migrations.
✓	Closed Issue #4	Menu Management	Implemented through Merged PR #55 , with nested menu item CRUD endpoints under restaurants and owner/admin write permissions. Merged PR #52 was reverted by Merged PR #54 , then replaced by Merged PR #55 .
✓	Closed Issue #19	Opening Hours	Implemented by the opening-hours work from Merged PR #59 , applied and cleaned through merged Merged PR #61 added opening-hour serialization, validation, and model string fixes.
✓	Closed Issue #17	Restaurant Photo	Implemented through Merged PR #70 , adding primary photo upload support to restaurant create/update flows and exposing primary photo URLs to public and owner screens.
✓	Closed Issue #18	Photo Gallery	Implemented through Merged PR #73 , adding the <code>RestaurantPhoto</code> model, gallery upload/list/delete endpoints, owner dashboard gallery management, and the restaurant detail photo carousel.

Epic: Reviews and Ratings System ([Closed Issue #9](#))

Done	Tracker	Feature	Description and PR evidence
✓	Closed Issue #8	Average Rating	Implemented through Merged PR #63 , adding signal-based recalculation of <code>average_rating</code> and <code>review_count</code> , plus rating sort support.
✓	Closed Issue #10	Review Replies	Implemented through Merged PR #65 , with a one-level depth guard, <code>POST /reviews/<id>/replies/</code> , route registration, and tests for auth, nesting, 404, and invalid reply depth.
✓	Closed Issue #11	Review Likes	Implemented through Merged PR #67 , with toggle behavior, reaction switching, <code>user_reaction</code> , DB-level helpfulness score, and helpfulness sorting.
✓	N/A	Review CRUD and restaurant-scoped review threads	Implemented through Merged PR #42 , with create/update/delete endpoints, ownership checks, paginated restaurant review lists, validation, reaction endpoints, frontend review UI, and 58 reported passing tests.
✓	N/A	Business owner answers	Implemented through Merged PR #65 and expanded in Merged PR #71 , using the one-level reply model and owner dashboard review-reply screens so restaurant owners can answer recent diner reviews.

Epic: User Authentication and Profile ([Closed Issue #12](#))

Done	Tracker	Feature	Description and PR evidence
✓	Closed Issue #35	Sign Up API and Flow	Implemented through Merged PR #40 , Merged PR #44 , Merged PR #48 , and the reusable auth module from Merged PR #74 , covering API guards, role-aware signup paths, and shared frontend auth forms.
✓	Closed Issue #36	Sign In API and Session/Token Flow	Implemented through Merged PR #40 , Merged PR #44 , Merged PR #42 , and Merged PR #74 , covering auth guards, sign-in pages, session storage, account navigation, and role-based redirects.
✓	Closed Issue #15	Restaurant Owner Role	Implemented through Merged PR #48 , Merged PR #71 , and Merged PR #74 , adding owner-specific restaurant management, permission enforcement, analytics, listings, menu, hours, gallery, and review-reply dashboard areas.
✓	Closed Issue #13	Favorites List	Implemented through Merged PR #71 , adding favorite endpoints, favorite metrics, frontend favorite buttons, session-aware mutation hooks, and a favorite gallery on the reviewer profile.
✓	Closed Issue #14	User Profile	Implemented through Merged PR #71 , adding the reviewer profile page, account menu integration, profile sections/tabs, favorite restaurants, and review/profile API support.

Epic: Discovery and Homepage ([Closed Issue #23](#))

Done	Tracker	Feature	Description and PR evidence
✓	Closed Issue #16	Search	Implemented through Merged PR #39 , Merged PR #62 , Merged PR #72 , and Merged PR #74 , adding the UI foundation, search suggestions, ranking-aware discovery, and location-aware restaurant search.
✓	Closed Issue #47	Discovery map reviews and business answers	Implemented through Merged PR #62 and the review PR sequence, with selected-restaurant review panels, writing reviews, one-level comments, owner answers, and reactions.
✓	Closed Issue #20	Popular Ranking	Implemented through Merged PR #72 , ranking homepage and discovery restaurants by average rating and extending the restaurant query builder, service layer, and tests for ranked results.
✓	Closed Issue #22	Map Integration	Implemented through Merged PR #71 , adding map-backed exploration, Google Maps seed support, restaurant map cards, hover previews, and favorite-aware map/list interactions.

Epic: Search and Filtering (Closed Issue #37)

Done	Tracker	Feature	Description and PR evidence
✓	Closed Issue #5	Advanced Filtering	Implemented through Merged PR #66 , with category, city, price/price_range, and min_rating query parameters composed with AND logic.
✓	Closed Issue #7	Price Range Filter	Implemented through Merged PR #66 Merged PR #64 was a closed, unmerged attempt, while the merged solution landed in the advanced-filtering PR.
✓	Closed Issue #6	Location Filter	City filtering was added in Merged PR #66 and completed with geospatial/location-aware search in Merged PR #74 , including the geospatial search module, discovery endpoint cleanup, and frontend location search flow.

Cross-Cutting Infrastructure and Reliability

Done	Tracker	Feature	Description and PR evidence
✓	Closed Issue #24	Object File Management	Implemented through Merged PR #49 and Merged PR #50 , adding file storage services, local/MinIO backends, validation, thumbnail generation, stored-file indexing, obfuscated file URLs, and model integration.
✓	Closed Issue #21	Atomic Transactions	Implemented through Merged PR #68 , wrapping multi-step review, reaction, restaurant, and opening-hour operations in <code>transaction.atomic()</code> and adding rollback tests.
✓	N/A	Controller and module cleanup	Merged PR #74 split restaurant controllers into focused modules, moved authentication UI into a reusable Nx module, and reorganized the owner dashboard into route-level parts for listings, menu, reviews, overview, and analytics.
✓	N/A	API documentation and architecture baseline	Merged PR #28 , Merged PR #39 , Merged PR #48 , and Merged PR #60 shaped the PRD, UI foundation, DRF migration, OpenAPI docs, and serializer cleanup. Local <code>docs/TECHNICAL_SPECIFICATION.md</code> and <code>docs/DEVELOPMENT_PLAN.md</code> provide additional architecture and schedule references.

6. Challenges and Solutions

Several implementation problems shaped the final codebase.

- **Technical & Collaborational Skills Gap:** The teammates were required to be on onboarded on the way the project was going to be implement. So conducted meeting and group sessions about Git, Github flow, Django REST Framework, Next.js, and the project structure. We also talked about SDLC and software engineering best-practices This helped us to get on the same page and work more efficiently.
- **Aggregate rating correctness.** The backend needed to keep restaurant-level averages consistent while reviews, replies, and reactions were changing. The solution was to centralize the aggregation in the repository and update restaurant records from the review layer and use **signal** handlers to recalculate totals whenever reviews or reactions changed.
- **Reply and owner-answer boundaries.** The project needed one-level replies without turning the app into a generic comment system. The solution was to keep public replies scoped to the parent review and use owner-specific rules to mark business answers.
- **Multi-step write safety.** Menu and content updates involved multiple database operations. Wrapping those steps in `transaction.atomic()` prevented partial updates from leaving the data model in an inconsistent state.
- **Photo delivery.** Image uploads had to work across backend storage and frontend previews. The file-service abstraction and primary photo URL resolver made this portable.

7. Team Communication and Development Process

The team used GitHub as the coordination hub for the project. Feature work was tracked through GitHub issues, grouped by epic, and implementation work was submitted through pull requests so each change had a visible branch, discussion history, and merge record. This made the final report traceable: open issues represent remaining scope, while merged PRs represent completed implementation evidence.

We also held mentoring and coordination meetings focused on the practical workflow of Git, GitHub, and the software development lifecycle. These meetings covered how to create branches, make focused commits, push work, open pull requests, resolve merge conflicts, and connect PRs back to issues. They also covered SDLC habits such as breaking requirements into smaller deliverables, implementing one vertical slice at a time, testing before merging, and keeping documentation aligned with the codebase.

The project was built incrementally rather than as one large final integration. We started with repository setup, documentation, and application foundations; then added restaurant CRUD, authentication, owner permissions, reviews, file storage, filtering, and dashboard behavior in separate steps. Each increment was meant to leave the application closer to a demoable state, which reduced integration risk and made it easier to review each teammate's contribution.

The resulting workflow was:

- Plan features as issue-tracker items and assign them to an epic.
- Create a branch for a small, reviewable change.
- Implement the backend, frontend, or documentation slice needed for that issue.
- Open a pull request with the linked issue and implementation summary.
- Review, test, and merge the PR before building the next increment.
- Update the report and diagrams from the merged implementation history.

8. Contribution Summary

Individual contributions are reported from merged PR authorship rather than commit-count guessing. Closed but unmerged PRs are mentioned only when their work was later absorbed by a merged PR.

Mardin / KrrOption

Feature group / title	Merged PRs
Project and infrastructure foundation	Merged PR #1 , Merged PR #25 , Merged PR #28 , Merged PR #39 , Merged PR #69
Core restaurant and owner-management backend	Merged PR #40 , Merged PR #48 , Merged PR #60
Review API and frontend integration	Merged PR #42
Object file management and image delivery	Merged PR #49 , Merged PR #50 , Merged PR #70
Menu-management finalization and revert cleanup	Merged PR #54 , Merged PR #55
Discovery and homepage UI	Merged PR #62
Favorites, reviewer profile, and map integration	Merged PR #71
Popular ranking and homepage discovery ordering	Merged PR #72
Restaurant photo gallery and owner gallery management	Merged PR #73
Location-aware search, controller cleanup, and frontend module extraction	Merged PR #74

Mardin's work carried the system skeleton, API modernization, frontend architecture, owner-facing restaurant management, object storage, public discovery UI, primary-photo and gallery experiences, favorites/profile flows, map-backed exploration, ranking, and final location-search/module-cleanup work.

Arsha / Arsha-shf

Feature group / title	Merged PRs
Restaurant update endpoint continuation	Merged PR #46
Category many-to-many migration	Merged PR #56
Average rating aggregation	Merged PR #63
Review replies	Merged PR #65
Review likes, dislikes, helpfulness score, and user_reaction	Merged PR #67
Atomic transaction safety	Merged PR #68

Arsha's work concentrated on data correctness and review-system depth: category relationships, rating aggregates, reply boundaries, reaction behavior, and rollback-safe multi-step writes.

TugraDuygulu / TugraDuyguLu

Feature group / title	Merged PRs
Health endpoint uptime improvement	Merged PR #32
Restaurant delete endpoint	Merged PR #43
API auth and role guard decorators	Merged PR #44
Menu item CRUD initial merged implementation	Merged PR #52
Advanced restaurant filtering	Merged PR #66

TugraDuygulu's work contributed endpoint-level backend functionality, role-guard infrastructure, menu CRUD groundwork, and the final advanced filtering implementation for category, city, price, and minimum rating.

burak671

Feature group / title	Merged PRs
Health endpoint versioning	Merged PR #34
Restaurant creation implementation	Merged PR #41
Opening-hours integration cleanup	Merged PR #61

burak671's direct merged work covered health-version metadata and restaurant creation. The opening-hours contribution was first attempted in closed Closed PR #59 , then incorporated into main through merged Merged PR #61 .